Inside the National Research Platform

How we build it, how we run it, and how you can use it

Daniel Diaz · San Diego Supercomputer Center, UC San Diego

SCinet · SC26 · Chicago · November 2026



Agenda

- What NRP is, in five minutes
- How it's built: the network, sites, storage, ingress
- How we run it: GitLab and Argo CD, CI in VMs, NetBox, monitoring, perfSONAR, security, sharing GPUs
- AI: the LLM service, from the gateway to the GPUs, and an ops bot built on it
- Using it yourself
- Live demo, then questions

Slides and the demo notebook:

https://training.nrp-nautilus.io/scinet-sc26

About 33 minutes of slides,
a 12-minute demo,
then questions.

Ask anything along the way.

What NRP Is

What is NRP?

- One national **Kubernetes** cluster, **Nautilus**, for research and education
- Operated by SDSC (UC San Diego), Nebraska–Lincoln and MGHPCC; funded by NSF

https://dash.nrp-nautilus.io



Nodes, GPUs, cores and storage: cluster snapshot, 10 Nov 2026

What is NRP?

- One national **Kubernetes** cluster, **Nautilus**, for research and education
- Operated by SDSC (UC San Diego), Nebraska–Lincoln and MGHPCC; funded by NSF
- Hardware from 70+ institutions, on research and education networks
- Today:
  - 536 nodes at 129 sites
  - 1,460 GPUs in 33 models; 59,352 CPU cores
  - 15 PB of raw Ceph storage

https://dash.nrp-nautilus.io



Sites from the NRP dashboard, 10 Nov 2026
Not drawn: 8 sites in Hawaii, Guam, Korea, Brazil and Europe

Nodes, GPUs, cores and storage: cluster snapshot, 10 Nov 2026

What is NRP?

- One national **Kubernetes** cluster, **Nautilus**, for research and education
- Operated by SDSC (UC San Diego), Nebraska–Lincoln and MGHPCC; funded by NSF
- Hardware from 70+ institutions, on research and education networks
- Today:
  - 536 nodes at 129 sites
  - 1,460 GPUs in 33 models; 59,352 CPU cores
  - 15 PB of raw Ceph storage
- Free for non-commercial research and education
- One team runs it from the OS up; campuses keep priority on what they contribute

Nodes, GPUs, cores and storage: cluster snapshot, 10 Nov 2026

https://dash.nrp-nautilus.io



Sites from the NRP dashboard, 10 Nov 2026
Not drawn: 8 sites in Hawaii, Guam, Korea, Brazil and Europe

From PRP to NRP

2015

Pacific Research Platform

Data transfer nodes (FIONAs) in campus Science DMZs, on CENIC and Pacific Wave

OAC-1541349

2018

Toward a National Research Platform

The same model, beyond the West Coast

OAC-1826967

2021

National Research Platform

An NSF Category II resource: GPUs, storage and services for research and education

OAC-2112167

2026

Today

One Kubernetes cluster, Nautilus: 536 nodes at 129 sites, and AI models as a service

Under each milestone: its NSF award.

NRP in Numbers

536
nodes

129
sites on the dashboard

1,460
GPUs in 33 models

59,352
CPU cores

14.9 PB
raw Ceph in 9 clusters

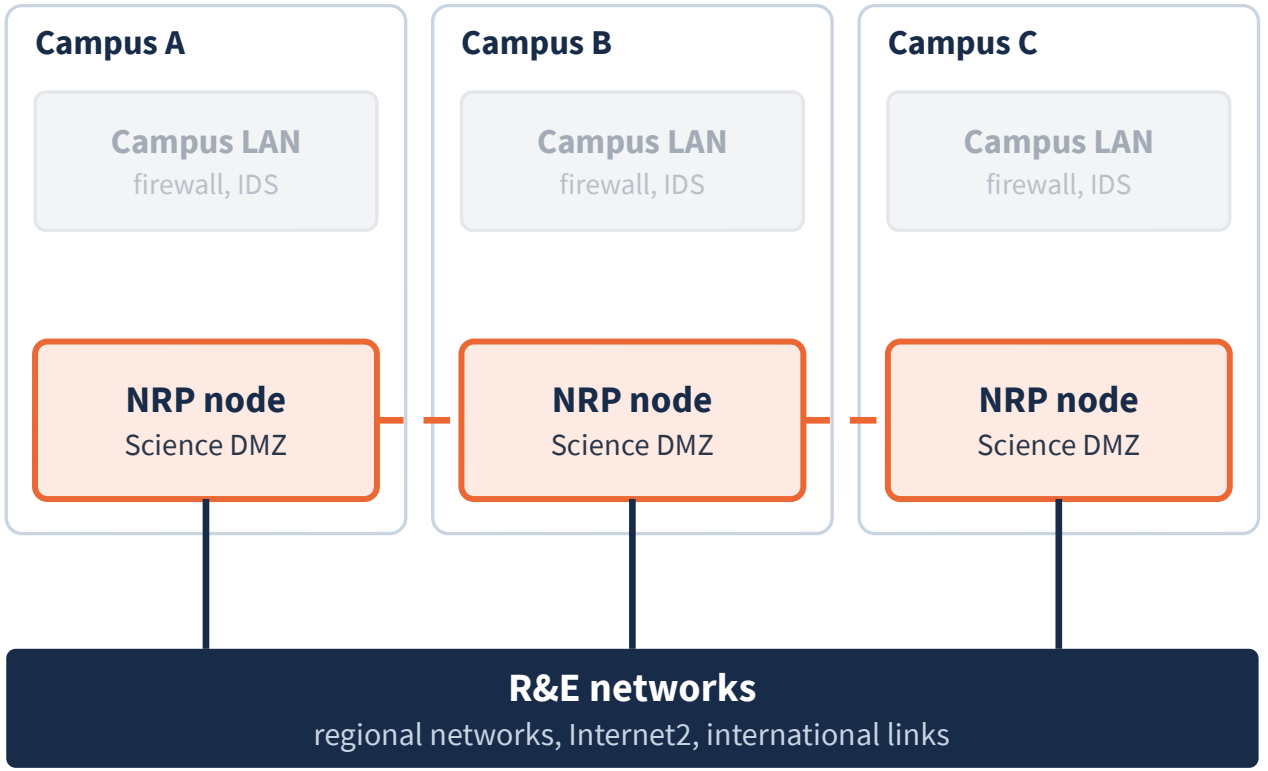
2,334
namespaces: labs, courses, services

Cluster snapshot, 10 Nov 2026 (Kubernetes v1.34.11). Sites: the NRP dashboard.

How It's Built

The Network Underneath

- Nodes sit in campus **Science DMZs**, outside campus firewalls, at 10–100 Gb/s
- **Jumbo frames:** MTU 9000 between all nodes
- Regional and national R&E networks carry the traffic
- 54 nodes sit in 51 Internet2 locations
- **Calico** builds the pod network: a VXLAN overlay, IPv4 and IPv6 (309 nodes dual-stack)



Dashed: the Calico VXLAN overlay. Pods on any node reach pods on any other node directly.

Policy Lives in the Cluster

- Nodes are open to the world; **Calico GlobalNetworkPolicy** filters at the host, before NAT
- Hosts in a central network set reach each other with no filtering
- Campuses keep no per-node firewall rules; local iptables and firewalld stay off
- Why: the node population changes every week, faster than any campus change process

From the internet	Why
`22/tcp`	SSH, for NRP operators
`443/tcp`	HTTPS; opened on every node by cluster policy
`ICMP`	ping, and path MTU discovery for jumbo frames
`8443/tcp`	object reads over HTTPS and XRootD (data caches)
`8444/tcp`	Pelican tokens, director health checks, metrics
`50000–51000/tcp`	GridFTP data channels

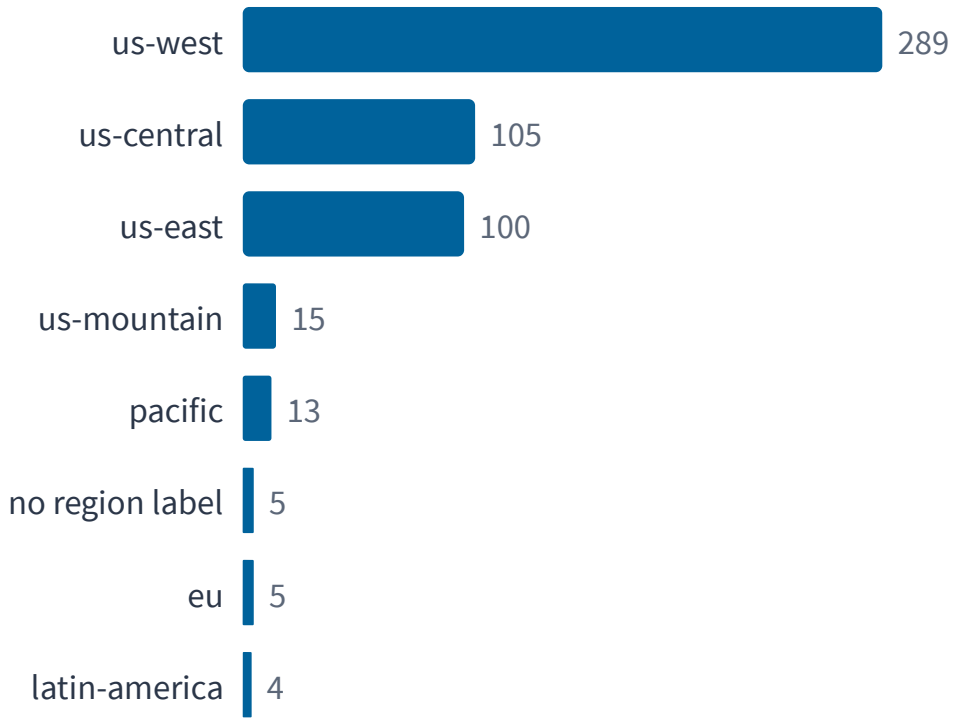
The last three only on nodes that run a data cache, an origin or a Globus endpoint.

https://nrp.ai/documentation/admindocs/participating/network/

One Cluster, Many Sites

- One control plane; Kubernetes 1.34
- Topology labels on every node: region, zone (143 zones), site
- Workloads ask for locality: spread over zones, or stay near their data
- Mostly x86-64; 3 ARM64 nodes
- Operated from SDSC, Nebraska–Lincoln and MGHPCC

Nodes by region

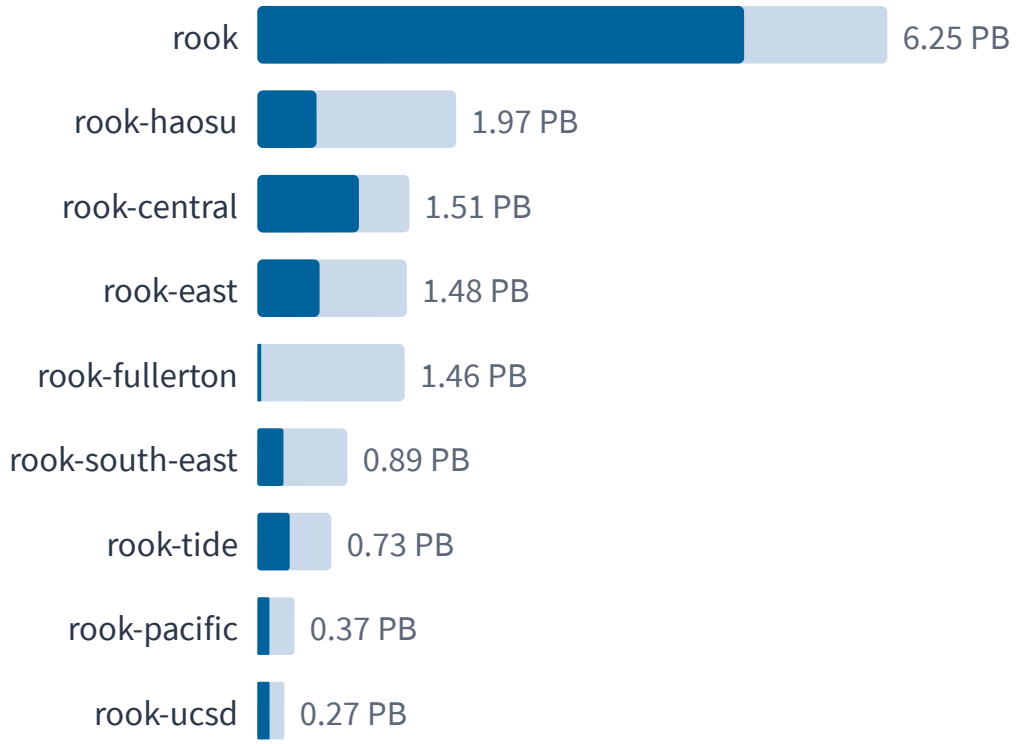


`topology.kubernetes.io/region`, 10 Nov 2026

Storage

- **Rook** runs 9 **Ceph** clusters, one per region or large site: 14.9 PB raw
- CephFS for shared files, RBD for block, S3 for objects
- **Linstor** (DRBD) for fast replicated block storage
- **CVMFS** for software; local NVMe for scratch
- Pick the pool near your compute

Ceph clusters: raw capacity, and used



From the CephCluster resources, 10 Nov 2026

More Than GPUs

Accelerators

NVIDIA GPUs
1,460 in 33 models, GTX 1080 Ti to H200

AMD GPUs
ROCm device plugin

Qualcomm Cloud AI 100
inference cards, requested like a GPU

Xilinx Alveo FPGAs
Vivado and Vitis

ESnet SmartNIC
FPGA network cards, P4 pipelines

ARM64 nodes
they need multi-arch images

Runtimes and networking

Containers
runc, plus the NVIDIA runtime

Kata VMs
a lightweight VM per pod

Sysbox
system containers: Docker inside a pod

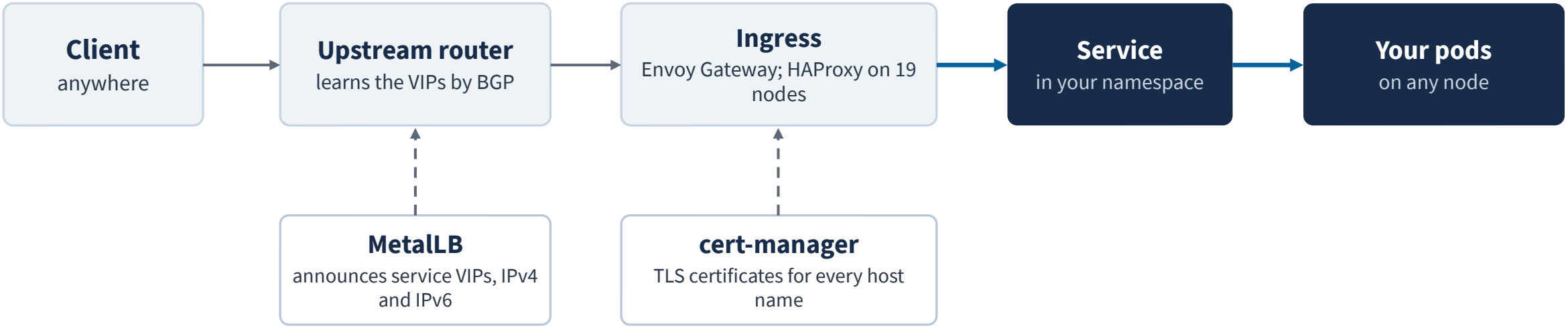
KubeVirt
full VMs: Ubuntu, Windows, Talos

Multus and SR-IOV
extra NICs in a pod, for network research

Confidential VMs
Kata with AMD SNP or Intel TDX, GPUs included

Runtime classes on the cluster: 29, most of them Kata variants (QEMU, Cloud Hypervisor, Firecracker, Dragonball).

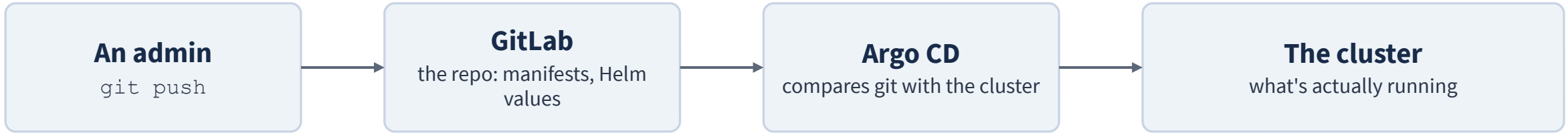
Getting Traffic In



- 1,006 Ingress objects on HAProxy, and 33 Gateway API routes on Envoy Gateway, which newer services use
- Users expose a web service with an Ingress or an HTTPRoute in their own namespace; DNS and TLS come with it
- Load-balancer VIPs for non-HTTP services, by request

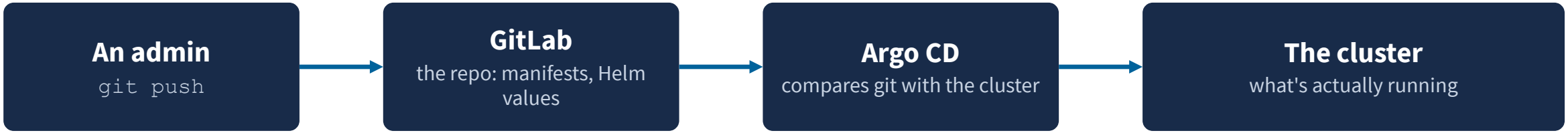
How We Run It

Git Is the Source of Truth



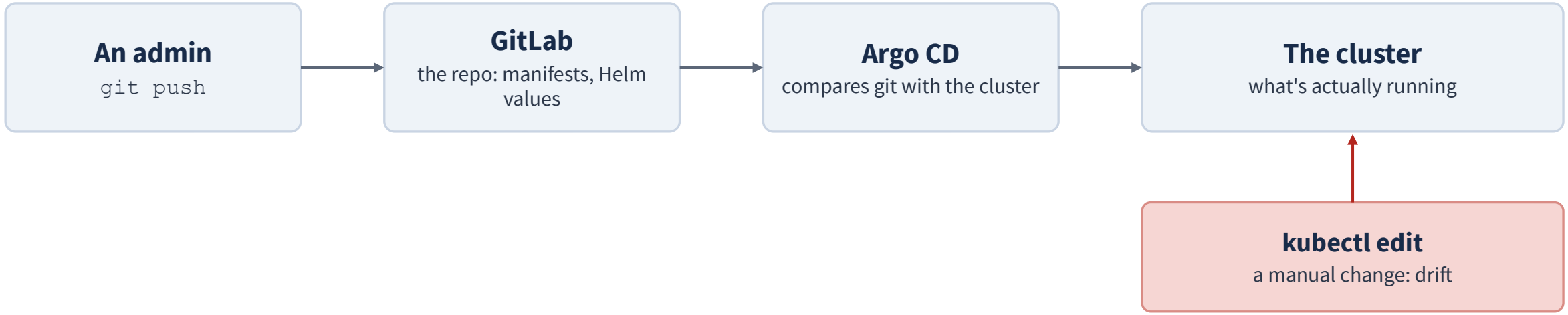
- Self-hosted **GitLab** 19.5.4: code, issues, a container registry, CI
- **Argo CD** syncs 20 applications, GitLab itself among them
- **Self-heal:** a manual change is reverted to what's in git; history and review come free
- Not everything is in Argo yet: some services are still applied by hand from their repos (`kubectl diff`, `kubectl apply`, then push)

Git Is the Source of Truth



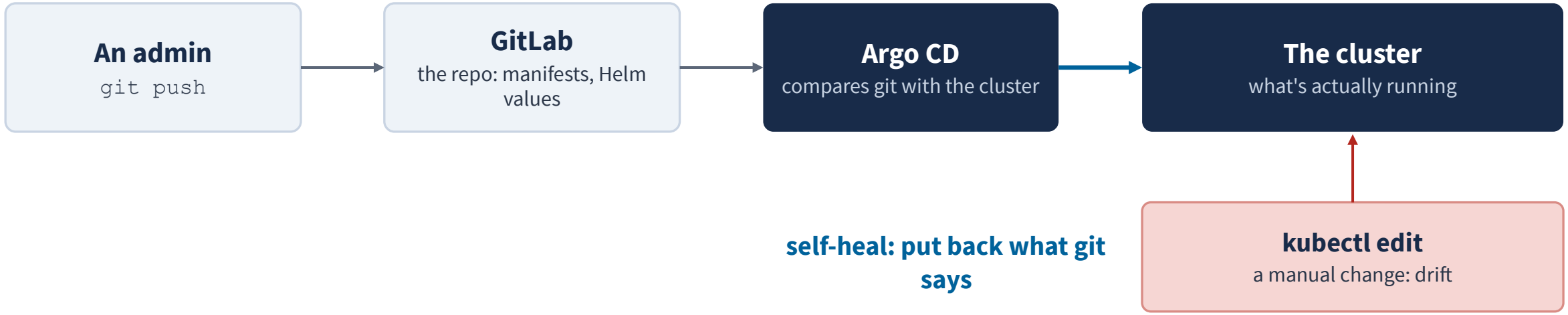
- Self-hosted **GitLab** 19.5.4: code, issues, a container registry, CI
- **Argo CD** syncs 20 applications, GitLab itself among them
- **Self-heal:** a manual change is reverted to what's in git; history and review come free
- Not everything is in Argo yet: some services are still applied by hand from their repos (`kubectl diff`, `kubectl apply`, then push)

Git Is the Source of Truth



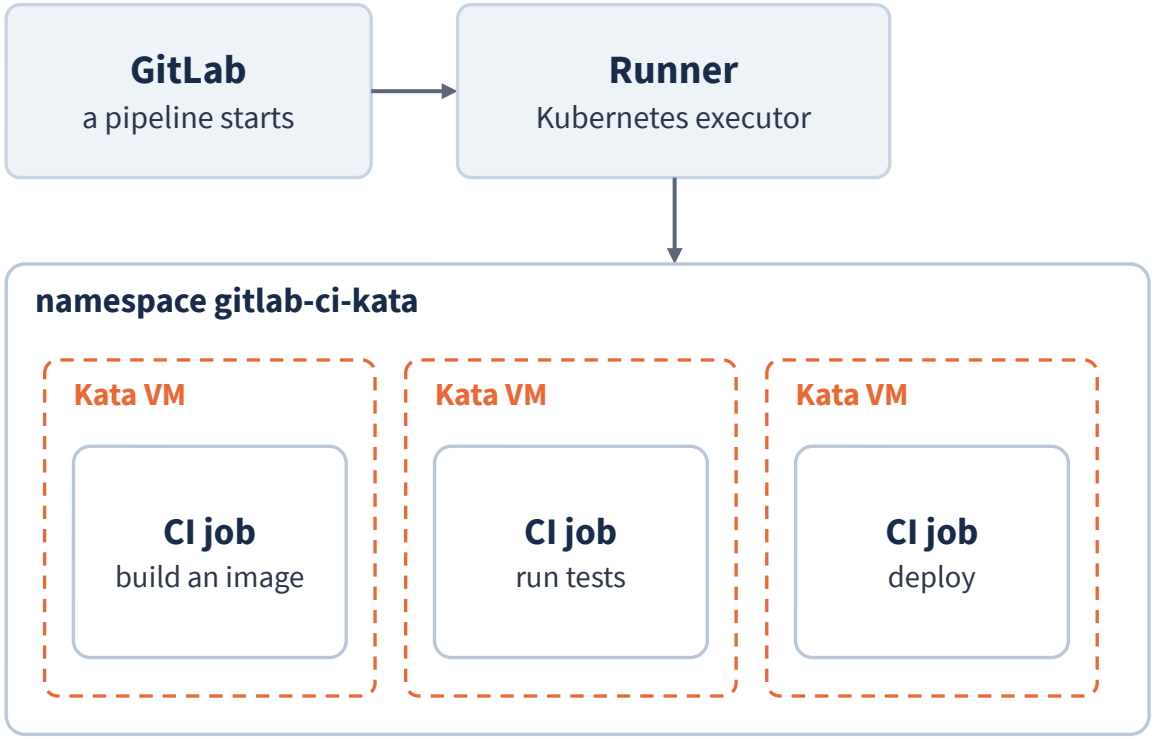
- Self-hosted **GitLab** 19.5.4: code, issues, a container registry, CI
- **Argo CD** syncs 20 applications, GitLab itself among them
- **Self-heal:** a manual change is reverted to what's in git; history and review come free
- Not everything is in Argo yet: some services are still applied by hand from their repos (`kubectl diff`, `kubectl apply`, then push)

Git Is the Source of Truth



- Self-hosted **GitLab** 19.5.4: code, issues, a container registry, CI
- **Argo CD** syncs 20 applications, GitLab itself among them
- **Self-heal:** a manual change is reverted to what's in git; history and review come free
- Not everything is in Argo yet: some services are still applied by hand from their repos (`kubectl diff`, `kubectl apply`, then push)

CI in a VM per Job



Runner config (from the cluster)

```
[[runners]]
  [runners.kubernetes]
    namespace = "gitlab-ci-kata"
    runtime_class_name = "kata"
    privileged = false
    image = "alpine:3.22"
    cpu_limit = "4"
    cpu_limit_overwrite_max_allowed = "12"
    memory_limit = "8Gi"
    memory_limit_overwrite_max_allowed = "50Gi"
```

- The shared runners start each CI job as a pod in its own lightweight VM: a privileged build can't reach the node
- 4 runners; images go to **gitlab-registry.nrp-nautilus.io** and are pulled from inside the cluster

Build and Deploy Your Own

.gitlab-ci.yml (from the NRP docs)

```
image: ghcr.io/osscontainertools/kaniko:debug

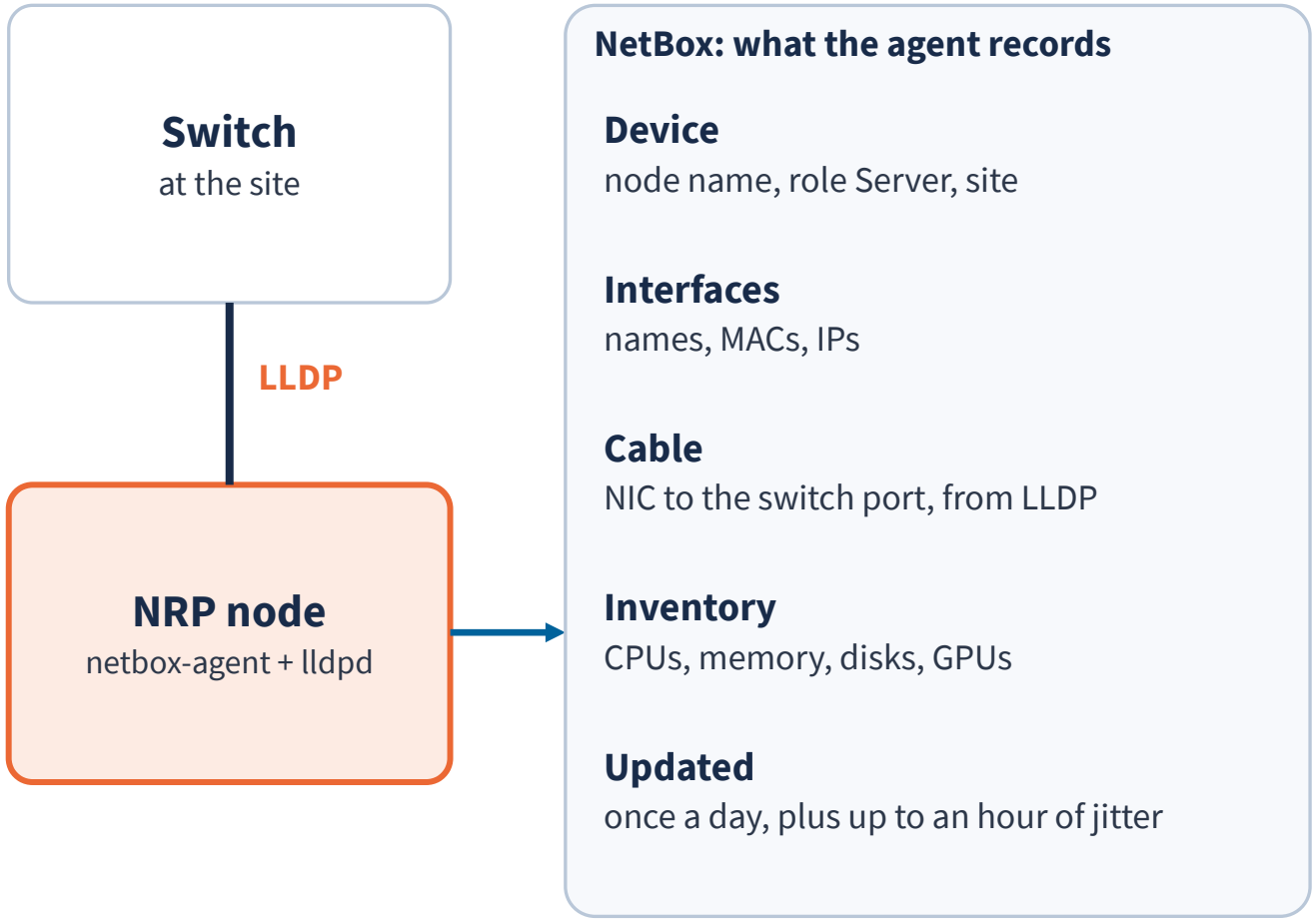
build-and-push-job:
  stage: build
  variables:
    GODEBUG: "http2client=0"
  script:
    - echo "{\"auths\":{\"$CI_REGISTRY\":{\"username\":
        \"$CI_REGISTRY_USER\",\"password\":\"$CI_REGISTRY_PASSWORD\"}}}"
        > /kaniko/.docker/config.json
    - /kaniko/executor --cache=true --push-retry=10
        --context $CI_PROJECT_DIR
        --dockerfile $CI_PROJECT_DIR/Dockerfile
        --destination $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
```

- Any NRP user can create projects at **gitlab.nrp-nautilus.io**
- Kaniko builds an image with no Docker daemon
- The registry is in the cluster: pulls are fast
- The GitLab agent for Kubernetes deploys to your namespace
- Build multi-arch images for the ARM64 nodes

https://nrp.ai/documentation/userdocs/development/gitlab/

NetBox Knows Every Node

- **NetBox 4**, deployed by Argo CD from the NetBox Labs Helm chart
- **netbox-agent** on every node (529 pods): privileged, host network
- **lldpd** learns the switch and port each NIC is cabled to
- The site comes from the node's `netbox.io/site` label
- Alongside: Diode for ingest, and a Nornir pod for automation



The Agent, Up Close

netbox_agent.yaml (token left out)

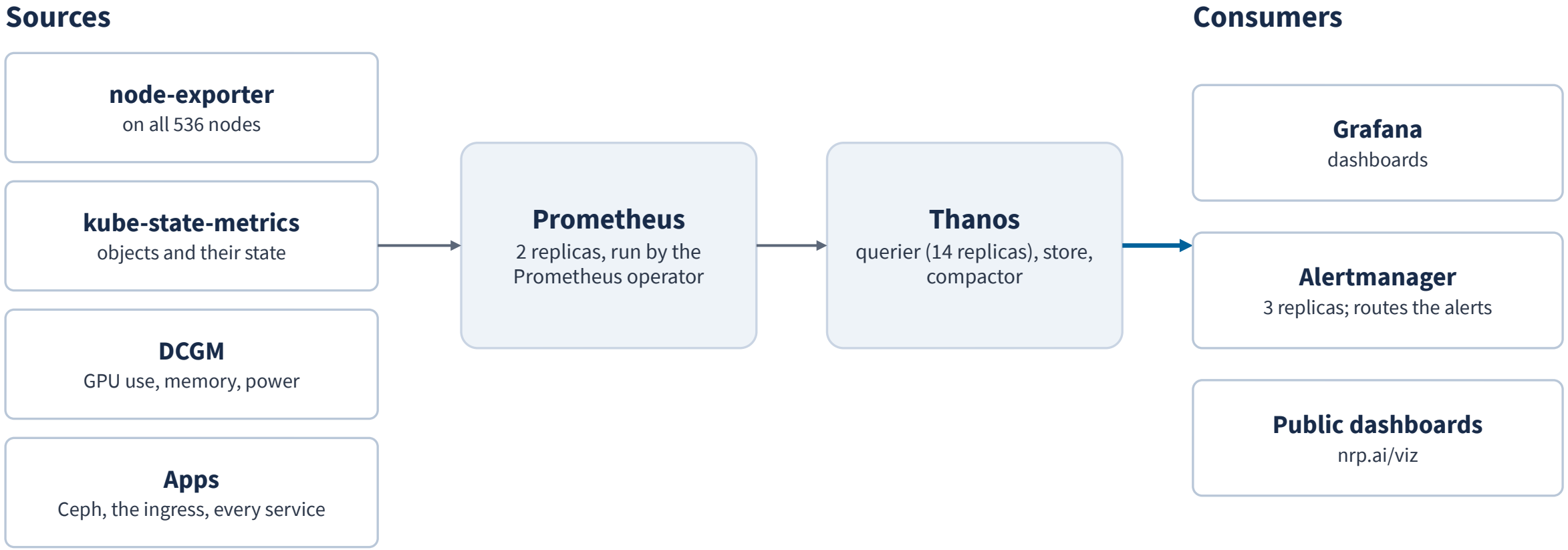
```
netbox:
  url: https://netbox-4.nrp-nautilus.io
network:
  ignore_interfaces: (dummy.*|docker.*|tunl*|cali*|vxlan.*|lo|ibp.*|.*ifb.*)
  ignore_ips: (127\.0\..*)
  lldp: true
  nic_id: name
  primary_mac: permanent
virtual:
  cluster_name: nautilus
datacenter_location:
  driver: cmd:kubectl get node ${NODE_NAME} -o jsonpath=
    '{.metadata.labels.netbox\.io/site}' | tr '[:upper:]' '[:lower:]'
inventory: true
```

The run loop (from the startup script)

```
lldpd           # start listening for LLDP
# at least 30 s so LLDP has found its neighbours
pause $(random_between 30 "$INITIAL_DELAY_MAX")
while true; do
  netbox_agent -c "$RUNTIME_CONFIG" --update-all -d &&
  netbox_agent -c "$RUNTIME_CONFIG" --update-inventory -d
  # a day, plus up to an hour of jitter
  pause $((RUN_INTERVAL + $(random_between 0 "$RUN_JITTER")))
done
```

Interfaces the overlay creates (cali*, vxlan*, tunl*) are ignored, so NetBox holds the physical network only.

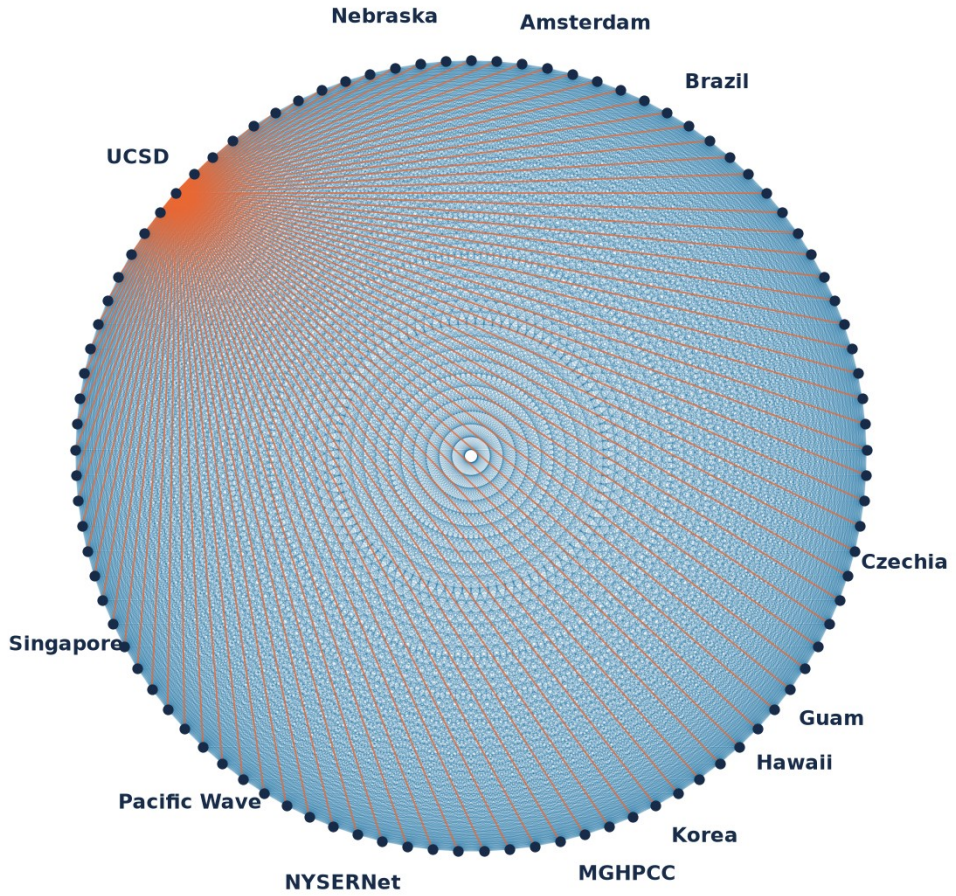
Watching the Cluster



Logs: Elastic collects syslog from every node, SSH logins included.

Measuring the Network

- A **perfSONAR testpoint** on every node (526 pods, from a DaemonSet)
- **MaDDash** all-to-all between one node per site: 97 sites, with the `nautilus.io/a2a-latency` label
- Mesh config, iperf3 servers and traceroute run as services in the cluster
- When a user says the cluster is slow, we can see whether it's the path
- perfsonar.nrp-nautilus.io



97 sites in the mesh, 10 Nov 2026; orange: one site's tests to every other

Security

Identity

- Authentik
- CILogon: federated campus login
- Groups drive namespace access

Detect

- Falco: runtime alerts, e.g. crypto miners in user pods
- Falco Talon: automated response
- Wazuh; Elastic syslog, SSH logins included

Prevent

- Gatekeeper admission policies
- Kata VMs for CI and untrusted builds
- Central Calico policy
- CVE mitigations rolled out by Argo CD

Audit

- IPMI scanner: alerts if a BMC is reachable from outside
- JupyterHub scanner: no anonymous hubs

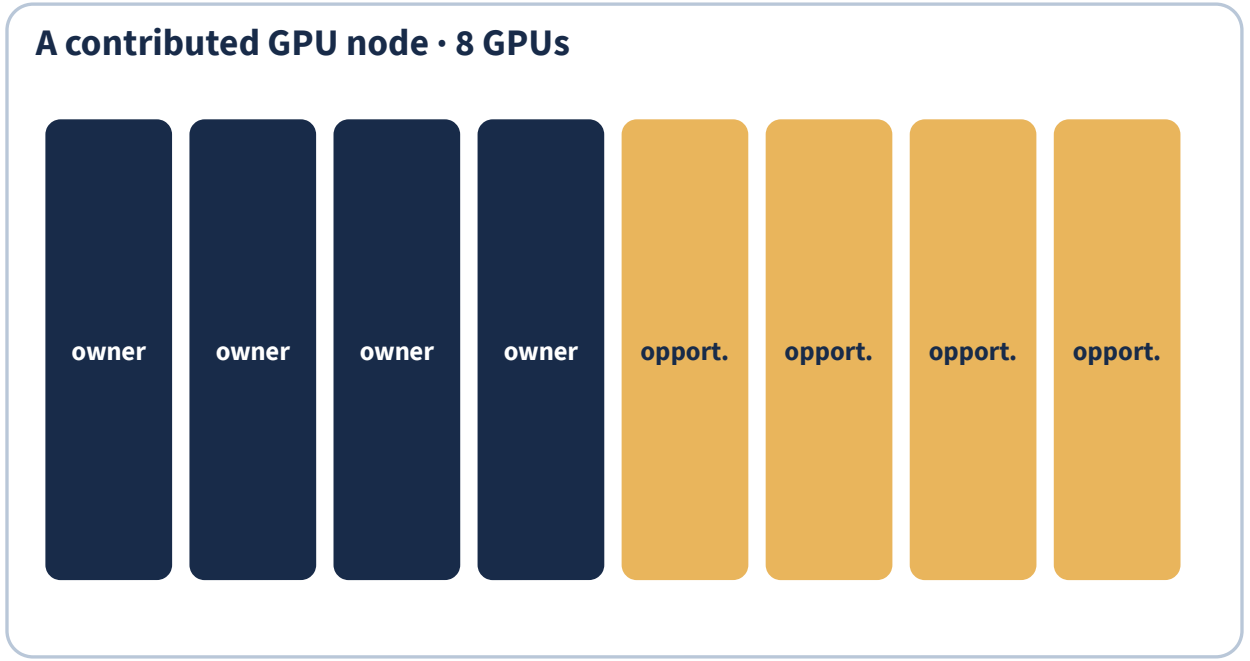
Routine Care

Tool	What it does
`kured`	Rolling reboots after kernel updates, a few nodes at a time
`self-node-remediation`	Fences and recovers nodes that stop responding
`node-label-monitor`	Watches node labels the scheduler depends on
`GPU operator, power management`	Drivers, and GPU power tracking
`Nagios NRPE`	Host checks
`Upgrade runbooks`	Kubernetes, Calico, Rook, GitLab, JupyterHub, Prometheus ... in the admin docs

https://nrp.ai/documentation/admindocs/upgrades/intro/

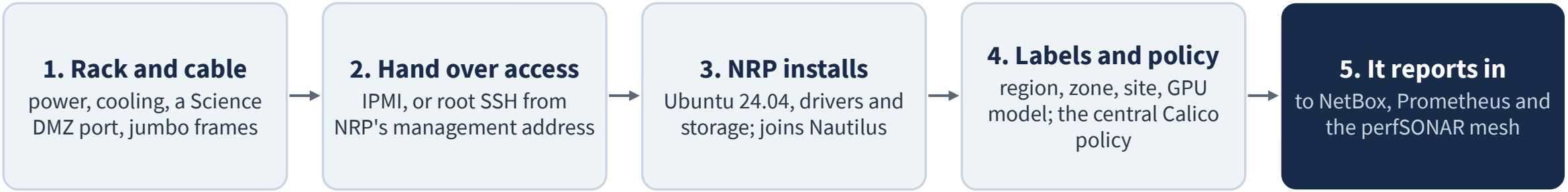
Sharing GPUs Fairly

- Contributors keep priority on their nodes: reservation taints, and Kueue with NRP's own controllers
- Priority classes: **owner**, **nice** (−10), **opportunistic** (preempted first)
- Idle GPUs go to opportunistic work, such as Open Science Grid jobs for IceCube and LIGO
- Opportunistic work must tolerate being preempted: checkpoint, or just restart



The owner submits more work: the scheduler preempts opportunistic pods to make room.

How a Node Joins



The site

- Keeps an on-site contact for hands-on work
- Sends maintenance notices; forwards warranty tickets

NRP

- Patching, packages and host hardening
- 24/7 monitoring and alerting; incident handling from the OS up

Then it's schedulable, yours first: owner priority for your users, idle time for everyone else.

https://nrp.ai/documentation/admindocs/participating/new-contributor-guide/

AI on NRP

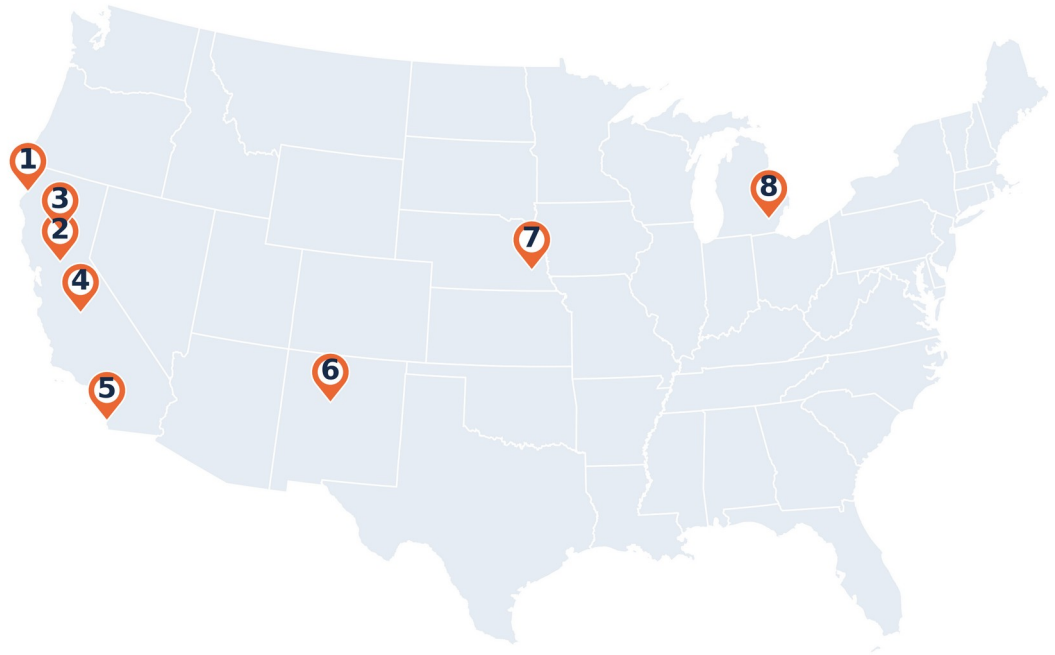
The LLM Service

- One OpenAI-compatible endpoint, **ellm.nrp-nautilus.io/v1**; an Anthropic-compatible one too
- 25 model names on that one endpoint
- 83 GPUs serve them, from A10s to H200s, at 9 sites
- About 7.1 billion tokens a day (September 2026)
- Free for research and education, with a token from nrp.ai/llmtoken

Model name	Serving on
`deepseek-v4-flash`	16 × H100
`glm-5`	8 × H200, 8 × H100
`kimi`	8 × A100
`minimax`	8 × A100
`qwen3`	8 × A100
`qwen3-small`	4 × A6000, 2 × H200
`gpt-oss`	2 × A6000
`gemma`	4 × A100, 1 × A6000
`gemma-small`	9 × A10
`gemma-small-e4b`	1 × A6000
`qwen3-embedding`	2 × L40, 2 × V100

Model-server pods by GPU, 10 Nov 2026. The catalog changes.

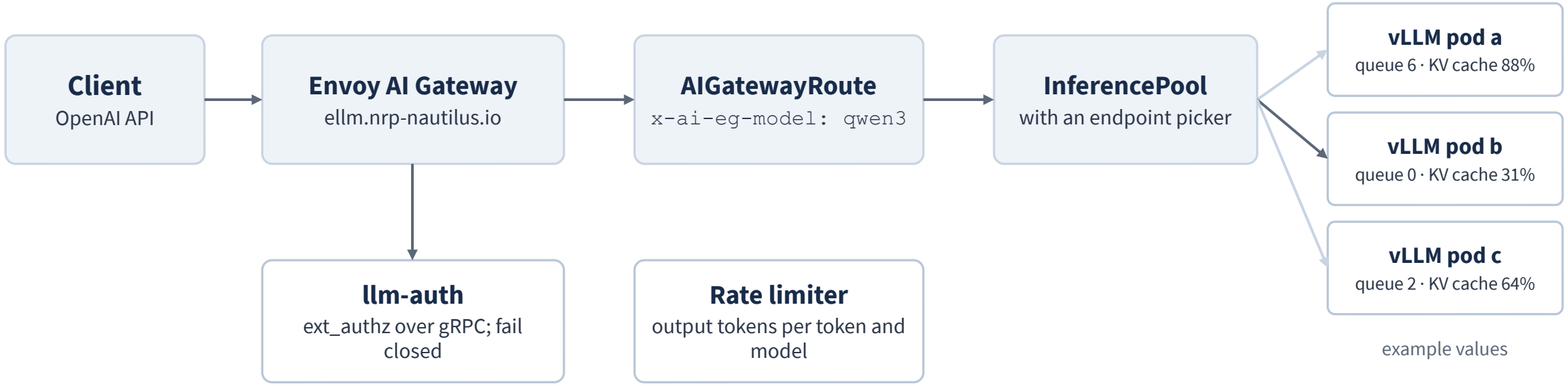
Where the Models Run



#	Site	Models	GPUs
1	Cal Poly Humboldt	qwen3-embedding	2 V100
2	Sacramento State	gemma-small-e4b, glm-5	4 H200, 1 A6000
3	CSU Chico	qwen3-small	4 A6000
4	Fresno State	qwen3-small	2 H200
5	UC San Diego + SDSU	deepseek-v4-flash, gemma, kimi, minimax, qwen3-embedding	20 A100, 16 H100, 2 L40
6	UNM	gemma	1 A6000
7	Nebraska–Lincoln	gemma-small, glm-5, gpt-oss	9 A10, 8 H100, 4 H200, 2 A6000
8	Internet2, Ann Arbor	qwen3	8 A100

Every pin is a set of model servers in the same cluster, behind the same endpoint. Fresno State and Internet2 Ann Arbor are placed by city.

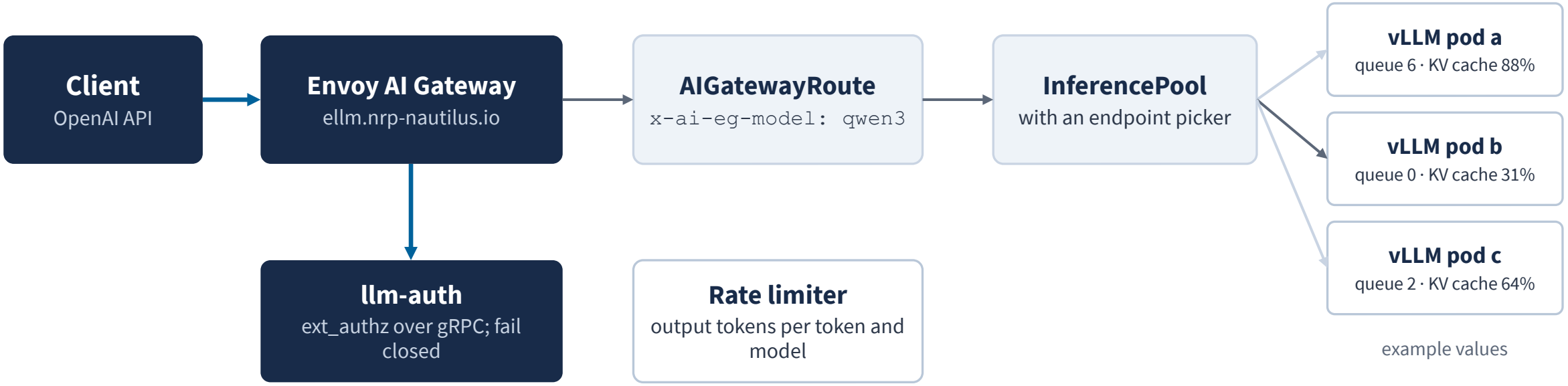
A Request, Step by Step



One request, five stops

The gateway is the Envoy Gateway from the ingress slide, plus its AI extension, Envoy AI Gateway. A client asks for `qwen3`.

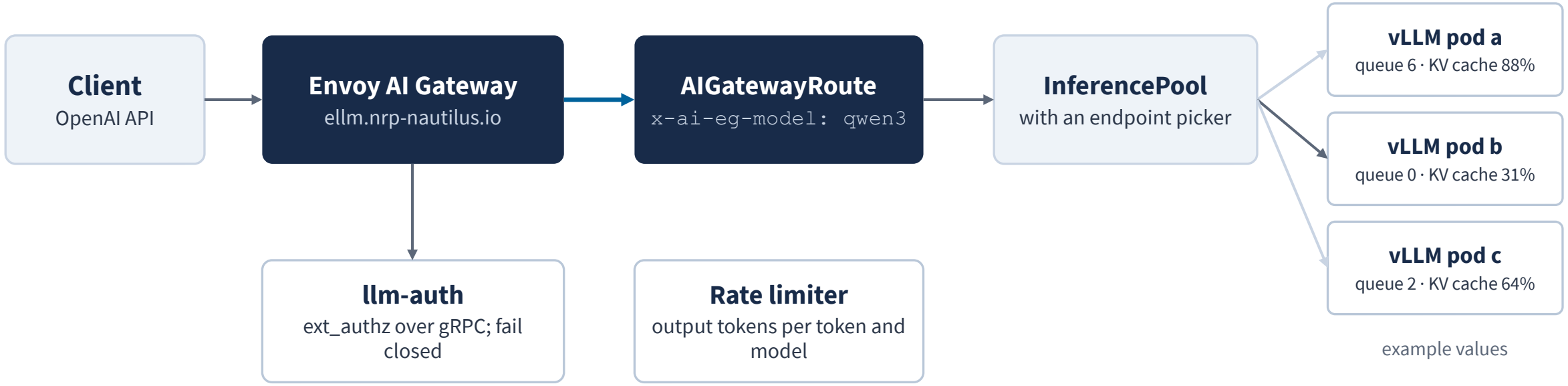
A Request, Step by Step



1. Who are you?

The gateway sends the bearer token to an external auth service over gRPC. If auth can't answer, the request fails: fail closed.

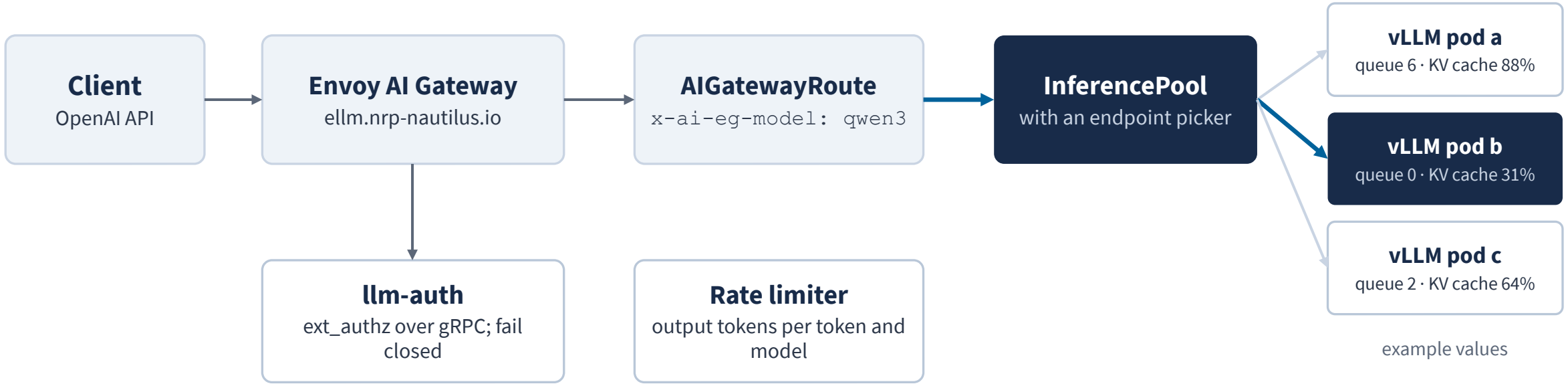
A Request, Step by Step



2. Which model?

The gateway reads `model` from the JSON body into the `x-ai-eg-model` header, and an AIGatewayRoute rule matches it to a backend: here an InferencePool.

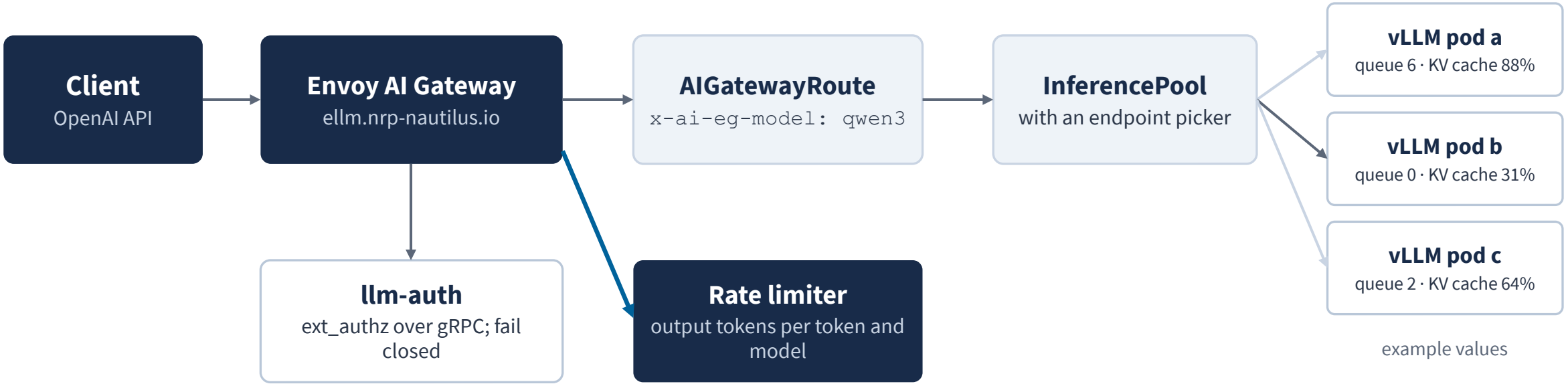
A Request, Step by Step



3. Which pod?

The endpoint picker scores the pool's vLLM pods by queue depth, KV-cache use and prefix-cache hits, and picks one.

A Request, Step by Step



4. The answer, and the bill

vLLM answers. The gateway reads the token counts from the response; the rate limiter charges output tokens to this token and model.

Gateway Policies

```
apiVersion: gateway.envoyproxy.io/v1alpha1
kind: BackendTrafficPolicy
metadata: {name: envoy-ai-gateway-nrp, namespace: nrp-llm}
spec:
  rateLimit:
    type: Global
    global:
      rules:
      - clientSelectors:
        - headers:
          - {name: authorization, type: Distinct}
          - {name: x-ai-eg-model, type: Distinct}
        limit: {requests: 48000, unit: Minute}
        cost:
          request: {from: Number, number: 1}
          response:
            from: Metadata
            metadata: {namespace: io.envoy.ai_gateway,
                       key: llm_output_token}
  retry:
    numRetries: 2
    retryOn: {httpStatusCodes: [500, 502, 503, 504]}
  healthCheck:
    active: {type: HTTP, http: {path: /health}, interval: 10s,
             unhealthyThreshold: 3}
    passive: {consecutive5XxErrors: 2, baseEjectionTime: 30s,
              maxEjectionPercent: 50}
```

Rate limit: 48,000 output tokens a minute, per token and model

Retries: twice, on 5xx

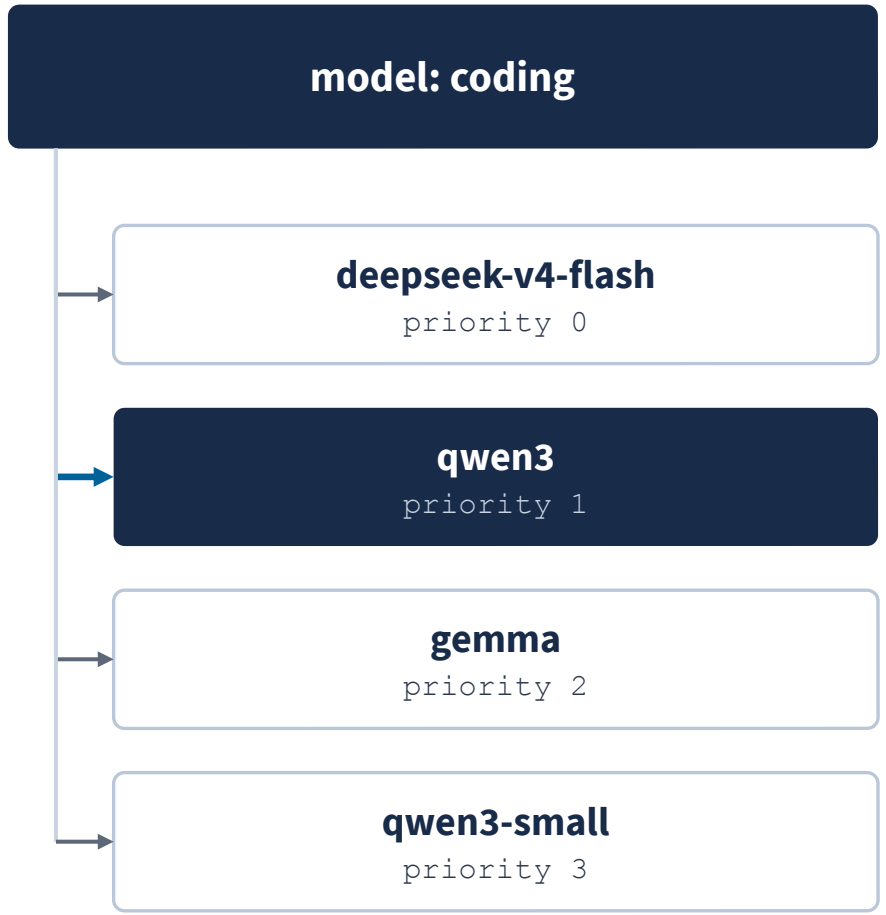
Health: probe /health every 10 s; eject a pod after two 5xx

What a client sees (demo, 10 Nov)

```
$ curl -si https://ellm.nrp-nautilus.io/v1/chat/completions \
    -H "Authorization: Bearer $OPENAI_API_KEY" \
    -d '{"model": "qwen3", ...}' | grep -i ratelimit
x-ratelimit-limit: 48000, 48000;w=60
x-ratelimit-remaining: 47999
x-ratelimit-reset: 9
```

Ask for a Task, Not a Model

Ask for	Tried in this order
agents	kimi → deepseek-v4-flash → qwen3 → qwen3-small
coding	deepseek-v4-flash → qwen3 → gemma → qwen3-small
general	deepseek-v4-flash → qwen3 → qwen3-small → minimax
reasoning	deepseek-v4-flash → minimax → qwen3-small → qwen3
vision	qwen3 → qwen3-small → kimi → deepseek-v4-flash
long-context	deepseek-v4-flash → gemma → kimi → minimax
kubernetes	kimi → gemma → qwen3 → qwen3-small
cuda	deepseek-v4-flash → qwen3-small → gpt-oss → gemma-small-e4b
service	deepseek-v4-flash → minimax → qwen3 → qwen3-small → gemma → gemma-small → gpt-oss



GitLab Duo Self-Hosted uses `service`.

Serving with vLLM

qwen3, from the cluster (shortened)

```
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: qwen38-flash-next-a100, namespace: nrp-llm}
spec:
  replicas: 2
  template:
    metadata:
      labels: {component: llm, service-group: qwen-vllm-inference}
    spec:
      priorityClassName: owner          # tolerates the reservation taint
      containers:
      - image: vllm/vllm-openai:v0.33.0
        args: ["vllm serve Qwen/Qwen3.8-Flash-Next-FP8 --port 5000
                --tensor-parallel-size 4 --enable-expert-parallel
                --enable-prefix-caching --max-model-len 1000000
                --kv-cache-dtype bfloat16 --otlp-traces-endpoint ..."]
        resources:
          limits: {cpu: "36", memory: 440Gi, nvidia.com/a100: 4}
        readinessProbe: {httpGet: {path: /health, port: 5000}}
      affinity:   # GPU memory > 80 GB, Ampere or newer
  volumeClaimTemplates:
  - metadata: {name: qwen38-flash-cache}
    spec: {storageClassName: linstor-igrok,
           resources: {requests: {storage: 256Gi}}}
```

Labels put the pod in the `qwen3` pool; priority **owner**

vLLM: tensor parallel 4, expert parallel, prefix caching, traces to OpenTelemetry

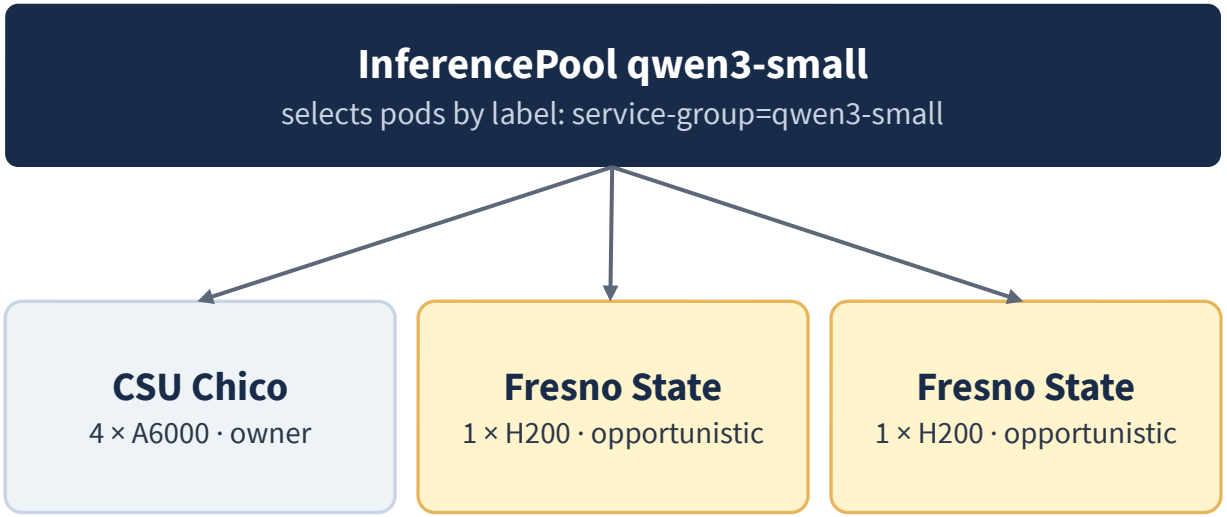
Health: probes on `/health`; vLLM and DCGM metrics go to the same Prometheus and Thanos, and drive the LLM status page

Weights on a Linstor volume, so a restart doesn't download them again

16 servers like this. Their config is in GitLab (`prp/llm-proxy`), applied by hand, not yet by Argo CD

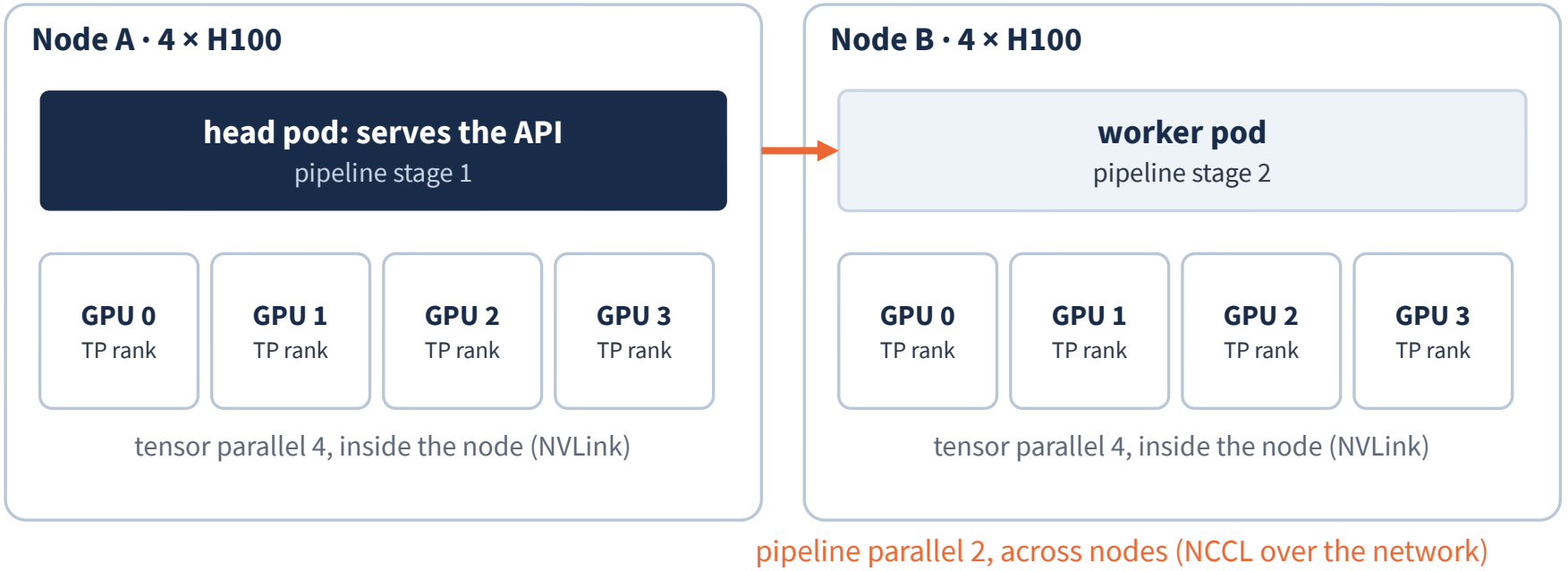
Borrowing Idle GPUs

- Extra replicas run at **opportunistic** priority on idle GPUs and join the same pool
- `qwen3-small`: 4 × A6000 at Chico State as owner, plus two H200 replicas at Fresno State as opportunistic
- `glm-5` and `gemma` borrow H100s and A100s at low priority too
- Preempted when the owner needs the GPU; the endpoint picker routes around the gap



Step 1: the H200's owner needs it back. One replica is preempted; the pool keeps answering from the other two.

One Model, Two Nodes



- DeepSeek V4.1 Flash, two copies
- Head and worker StatefulSets
- A custom supervisor restarts both together (LeaderWorkerSet isn't installed)
- Headless Service; NetworkPolicy limits traffic to the group
- NCCL settings pinned, e.g. `NCCL_SOCKET_IFNAME=eth0`

Expert parallelism spreads the model's experts across the GPUs. Both copies run on SDSC's Expanse H100 nodes.

Around the Models

Open WebUI
chat in the browser, with a terminals operator

Web search
SearXNG, for tools and chat

Embeddings
qwen3-embedding; TEI on a GPU

Vector databases
Milvus (managed) and Qdrant

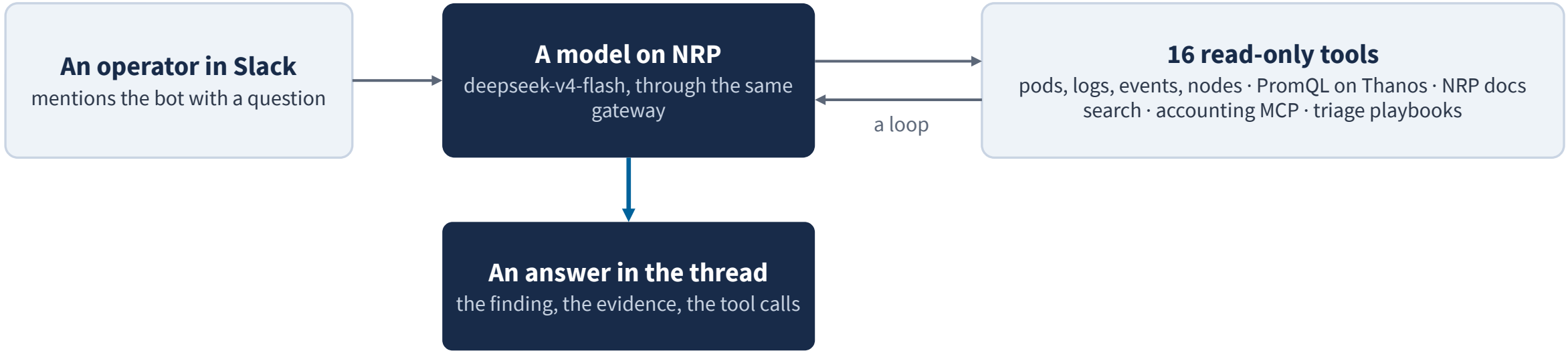
MCP route
an MCP gateway; an accounting MCP server

Telemetry
OpenTelemetry collector into ClickHouse

Coding agents
Claude Code, opencode and others, with tested configurations

All of it on the same cluster, most of it deployed the same way as everything else: from git.

An LLM on Call: nrp-ops-bot



Guardrails

- RBAC denies secrets, exec and every write
- Namespace allowlist; credentials redacted
- Logs and events treated as untrusted (prompt injection)
- Budgets on tool calls, iterations and time

Status

- Phase 1, read-only: built and tested with mocks
- Phase 2, write actions behind two-human approval: not started
- Also an MCP server, and an eval harness to compare models

Using It Yourself

Ways In

JupyterHub
notebooks with a GPU, in the browser

`jupyterhub-west.nrp-nautilus.io`

Coder
VS Code in the browser, in your namespace

`coder.nrp-nautilus.io`

kubectl
jobs, services and storage in a namespace

`nrp.ai/config`

GitLab
repos, CI in Kata VMs, a registry

`gitlab.nrp-nautilus.io`

LLM API
OpenAI- and Anthropic-compatible

`ellm.nrp-nautilus.io/v1`

Open WebUI
chat with every model

`nrp-openwebui.nrp-nautilus.io`

Who uses them today: research groups, each in a namespace; courses, each with its own JupyterHub; and collaborations through OSG.

Getting Access

1. Log in at **nrp.ai**

Log In → CILogon → your institution

2. Get into a namespace

Request one at **get-started.nrp-nautilus.io**

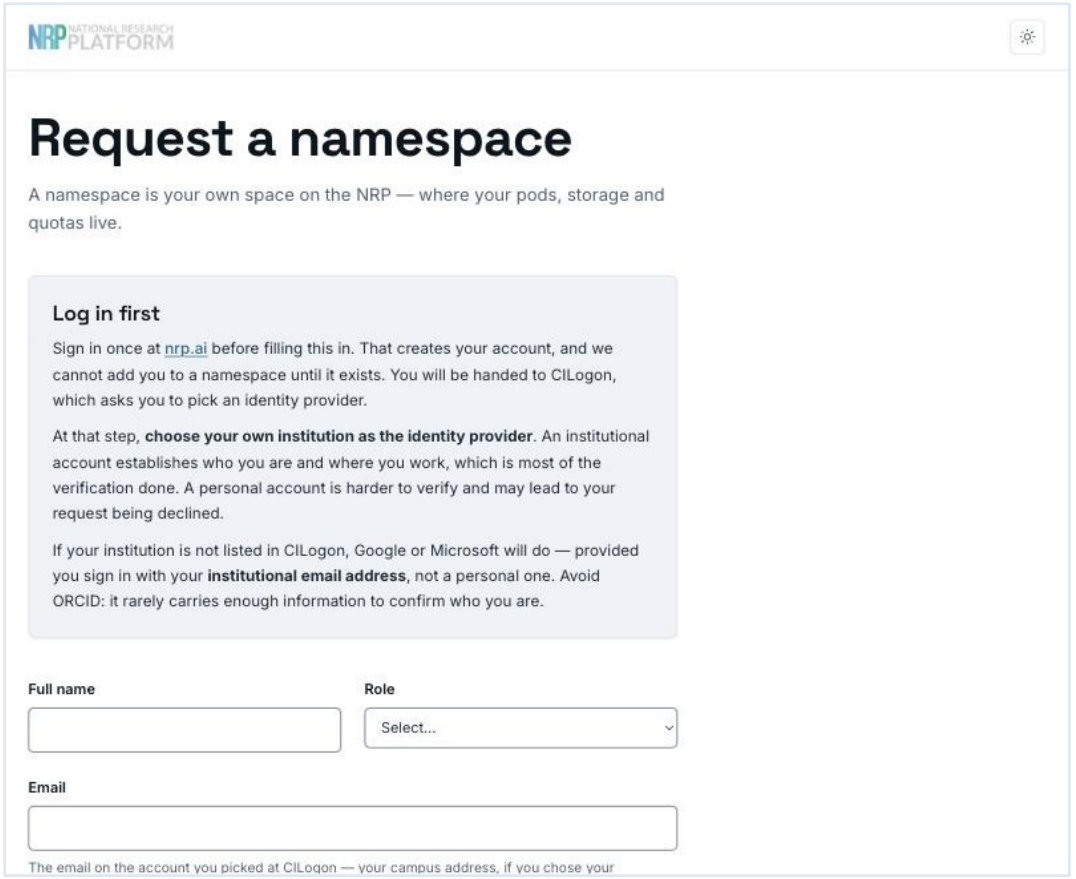
Or ask an existing namespace admin to add you

3. Start working

JupyterHub, kubectl, GitLab, or an LLM token from **nrp.ai/llmtoken**

Who can request a namespace: faculty, staff, postdocs and research scientists; students through their PI.

https://get-started.nrp-nautilus.io



Ways to Run Work

Interactive: JupyterHub

- Pick a GPU from the start menu; explore, debug, teach
- Idle servers stop: 1 hour after you close the browser on NRP's hub, 6 hours at most on any hub

Interactive: a pod

- Start a container with kubectl, then open a shell in it
- Deleted after 6 hours; at most 2 GPUs

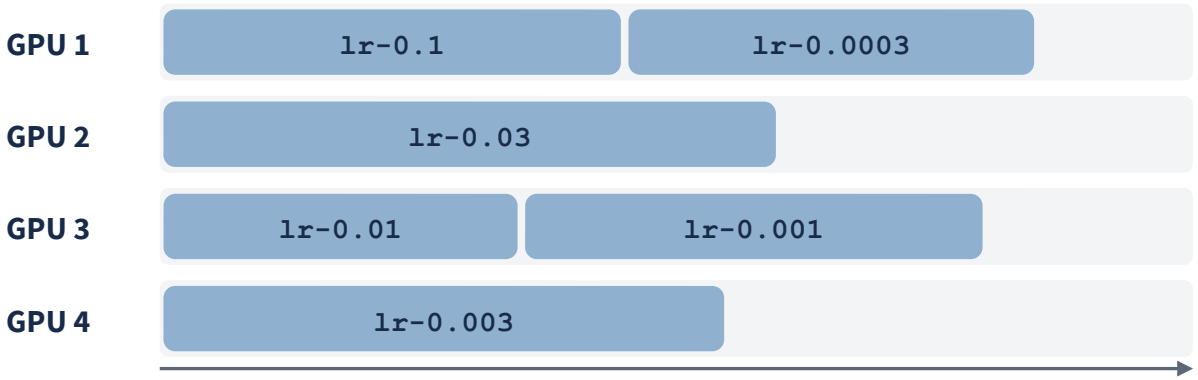
Long-running: a Deployment

- Keeps a service up (a web app, a database) and restarts it if it crashes
- Removed after 2 weeks unless admins grant an exception; no GPU if it sits mostly idle

Batch: a Job

- Runs your script to the end, then gives the GPU back
- Submit many at once; the rest wait in Pending for a free GPU

```
$ kubectl apply -f sweep/        # six training jobs
```



Six jobs on four GPUs: two wait in Pending and take the first GPUs that free up. When the last one ends, all four GPUs are free again.

A GPU Job

```
apiVersion: batch/v1
kind: Job
metadata: {name: gpu-train-daniel, namespace: my-lab}     # (1)
spec:
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: train
        image: pytorch/pytorch:2.9.0-cuda13.0-cudnn9-runtime  # (2)
        command: ["python", "train.py"]
        resources:                                          # (3)
          requests: {cpu: "2", memory: 6Gi, nvidia.com/gpu: 1}
          limits:   {cpu: "2", memory: 6Gi, nvidia.com/gpu: 1}
      affinity:                                             # (4)
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
            - matchExpressions:
              - {key: nvidia.com/gpu.memory, operator: Gt, values: ["20000"]}
              - {key: nvidia.com/gpu.compute.major, operator: Gt, values: ["7"]}
```

(1) **Where:** your namespace

(2) **What:** a container image, and the command to run

(3) **Resources:** 2 CPUs, 6 GB of memory, one GPU

(4) **Which GPU:** more than 20 GB, Ampere or newer; or name exact models

Where to Start

Use it

- **The LLM API in your automation:** config review, log triage, a docs assistant; task names keep working as the catalog changes
- **Your tools in a namespace:** web apps, Postgres and ClickHouse from the operators, S3 for data
- **GitLab CI** to build and ship them
- **GPUs for AI and ML:** a notebook to start, Jobs when it's real

Connect to it

- **Contribute nodes**, the way a node joins: you keep priority, NRP runs them
- **InterLink:** a Slurm cluster appears as a virtual node
- **Federation** with another Kubernetes cluster
- **FABRIC and SENSE** testbeds, with Multus for extra NICs
- **perfSONAR:** a testpoint on your node joins the mesh

https://nrp.ai/documentation/admindocs/participating/new-contributor-guide/

Live Demo

Acknowledgments

- NRP is supported by NSF awards OAC-1541349, OAC-1826967, OAC-2030508, OAC-1841530, OAC-2005369, OAC-2112167, CISE-1713149, CISE-2100237 and CISE-2120019
- Networks: CENIC, Pacific Wave, Internet2, MREN, GPN, NYSERNet, FLR, NEREN, Sun Corridor, OARnet, SCLR and the Albuquerque GigaPoP
- The NRP operations teams at SDSC, the University of Nebraska–Lincoln and MGHPCC, and the campuses that host and share their hardware
- SCinet, for having us during the NOC move

Questions?

Today's materials	`training.nrp-nautilus.io/scinet-sc26`
Portal and login	`nrp.ai`
Request a namespace	`get-started.nrp-nautilus.io`
LLM endpoint	`ellm.nrp-nautilus.io/v1`
LLM token	`nrp.ai/llmtoken`
perfSONAR	`perfsonar.nrp-nautilus.io`
Admin guide	`nrp.ai/documentation/admindocs`
Support chat and email	`nrp.ai/contact`

Daniel Diaz · d4diaz@ucsd.edu

Backup

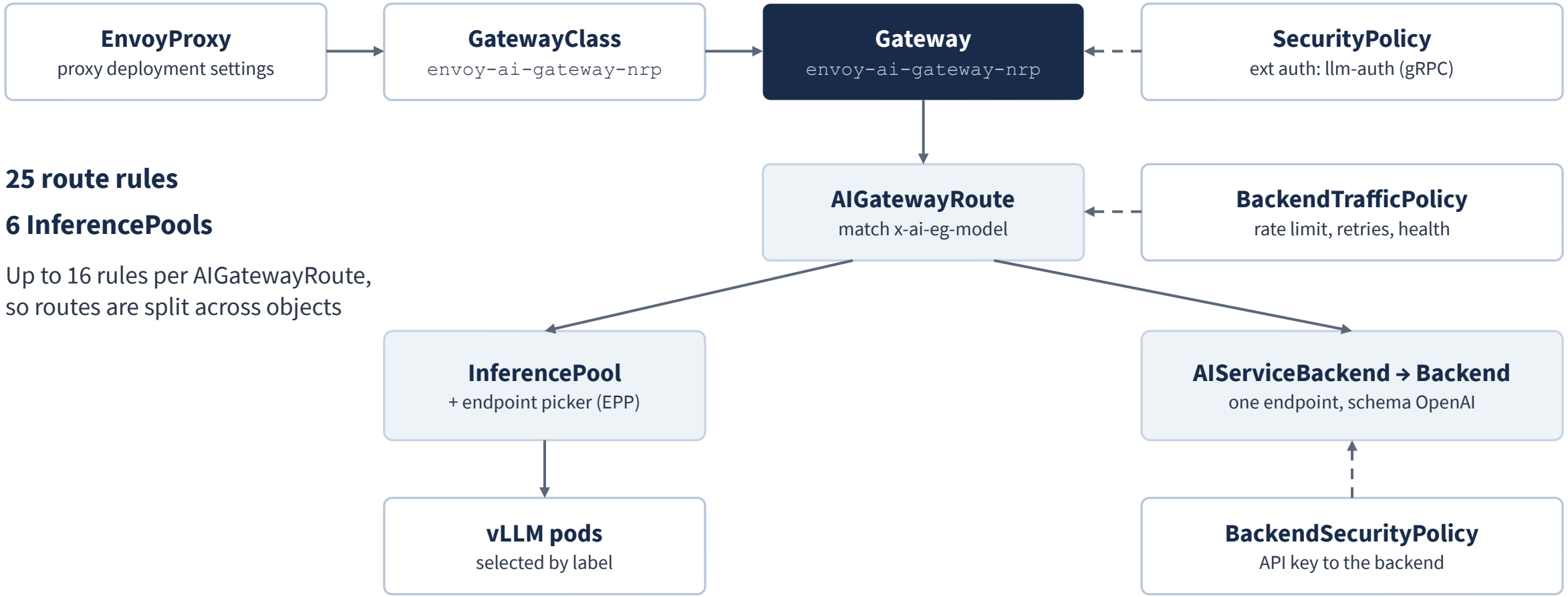
What Argo CD Runs

Application	Source	Sync
`cve-mitigations`	NRP GitLab: prp/cve-mitigations	auto
`falco`	Helm chart: falco	manual
`falco-talon`	Helm chart: falco-talon	auto, self-heal
`gitlab`	GitHub: nrp-nautilus/gitlab	auto, self-heal
`gpu-power-management`	GitHub: Sam6734/nrp-gpu-operator-tracking	auto, self-heal
`grafana`	Helm chart: grafana	auto, self-heal
`inspect`	NRP GitLab: nrp/inspect	auto, self-heal
`kueue`	GitHub: nrp-nautilus/kueue-gitops	manual
`kured`	Helm chart: charts	auto
`netbox`	Helm chart: netbox	auto, self-heal

Application	Source	Sync
`netbox-agent`	NRP GitLab: nrp/netbox-4	auto, self-heal
`nrp-aime`	GitHub: djw8605/nrp-aime	auto, self-heal
`nrp-clickhouse-deploy`	GitHub: djw8605/nrp-clickhouse	auto
`nrp-onboarding`	NRP GitLab: prp/nrp-onboarding	auto, self-heal
`nrp-ops-bot`	GitHub: djw8605/nrp-ops-bot	auto
`openwebui`	NRP GitLab: nrp/openwebui	auto
`ray`	NRP GitLab: prp/ray	manual
`self-node-remediation`	NRP GitLab: nrp/self-node-remediation	manual
`wazuh`	NRP GitLab: nrp/wazuh	auto, self-heal

Argo CD applications on the cluster, 10 Nov 2026; Argo's own example app left out.

Envoy AI Gateway Objects



25 route rules

6 InferencePools

Up to 16 rules per AIGatewayRoute, so routes are split across objects

From the NRP admin guide, Managing AI Models; object names from the cluster.

Using GPUs Fairly

Do

- Run batch work as a **Job** that exits
- Request what you'll use; keep limits within 20% of requests
- Keep data on a persistent volume or in S3
- Run a sweep as many one-GPU jobs
- Borrow idle A100/H100s with `priorityClassName: opportunistic`, and checkpoint

Don't

- Leave a notebook holding a GPU overnight
- Put `sleep` in a Job to keep a GPU (bannable)
- Ask for 4 GPUs "just in case": use must stay above 40%
- Keep results on the pod's own disk; it vanishes with the pod

https://nrp.ai/documentation/userdocs/start/policies/